Gaussian Process Latent Variable Modeling for Few-Shot Time Series Forecasting

Yunyao Cheng, Chenjuan Guo, Kaixuan Chen, Kai Zhao, Bin Yang, *Senior Member, IEEE*, Jiandong Xie, Christian S. Jensen, *Fellow, IEEE*, Feiteng Huang, and Kai Zheng, *Senior Member, IEEE*

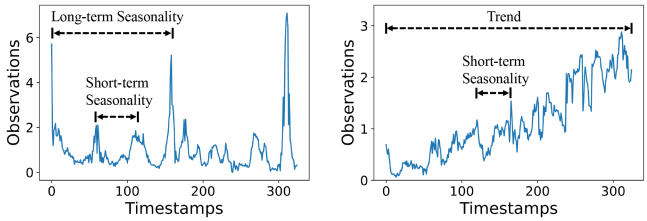
***Abstract*—Accurate time series forecasting is crucial for optimizing resource allocation, industrial production, and urban management, particularly with the growth of cyber-physical and IoT systems. However, limited training sample availability in fields like physics and biology poses significant challenges. Existing models struggle to capture long-term dependencies and to model diverse meta-knowledge explicitly in few-shot scenarios. To address these issues, we propose MetaGP, a meta-learning-based Gaussian process latent variable model that uses a Gaussian process kernel function to capture long-term dependencies and to maintain strong correlations in time series. We also introduce Kernel Association Search (KAS) as a novel meta-learning component to explicitly model meta-knowledge, thereby enhancing both interpretability and prediction accuracy. We study MetaGP on simulated and real-world few-shot datasets, showing that it is capable of state-of-the-art prediction accuracy. We also find that MetaGP can capture long-term dependencies and can model meta-knowledge, thereby providing valuable insights into complex time series patterns.**

***Index Terms*—Time series forecasting, few-shot learning, gaussian process, latent variable model, kernel association.**

I. Introduction

With the ongoing digitalization of societal and industrial processes and the increasing deployment of cyber-physical and IoT systems, massive time series data are being generated to enable value creation through a variety of analyses. Accurate time series forecasting can improve resource allocation, optimize industrial production processes, and enhance urban management.

However, not all datasets possess sufficient numbers of training samples; this is particularly prevalent in physics and biology data. Fig. 1 shows two samples from a real-world biological dataset, depicting the changes in microbial community sizes over time. Due to high collection and analysis costs, the number of samples of microbial community data is limited. When a model is transferred to a new data distribution and the new data has insufficient samples, the model cannot be fully trained, which can cause overfitting and decrease prediction accuracy. Therefore, this paper investigates the problem of how to achieve time series forecasting models that can be applied to new data with only a few samples. We refer to this as few-shot time series forecasting.



(a) Time series with both long-term and short-term seasonality (b) Time series with both trend and short-term seasonality

Fig. 1. Time series with varying patterns.

According to Bayesian theory, a well-performing model can be trained by enhancing likelihood (leveraging more data) or improving the prior (imposing constraints). Few-shot learning research focuses mainly on LLMs and hidden-state models: LLMs boost likelihood through large-scale pretraining, while hidden-state models refine priors with domain knowledge to improve generalization. However, LLMs struggle with scientific data due to their reliance on statistical learning over scientific laws, leading to poor generalization, violations of scientific principles, and difficulties with structured, multimodal, and low-resource datasets, exacerbated by scarce high-quality domain-specific data [1], [2]. Research on sequential latent variable models (LVMs) [3], [4], [5], [6] suggests that few-shot sequence forecasting can be improved by focusing on prior construction. Sequential LVMs can incorporate prior knowledge, for example, by making distributions of latent variables reflect domain-specific insights. Unlike autoregressive models, which establish correspondences between historical and future time series, sequential LVMs, as shown in Fig. 2(a), encode historical information as an initial state $\mathbf{z}_0$ and obtain subsequent latent states through a latent dynamic function, $\mathbf{z}_i = f(\mathbf{z}_{i-1})$. A predicted observation is derived from an emission function $\hat{\mathbf{y}}_i = g(\mathbf{z}_i)$. Therefore, sequential LVMs can infer the dynamics

Received 5 November 2024; revised 7 April 2025; accepted 10 May 2025. Date of publication 2 June 2025; date of current version 7 July 2025. Recommended for acceptance by J. Tang. *(Corresponding author: Chenjuan Guo.)*

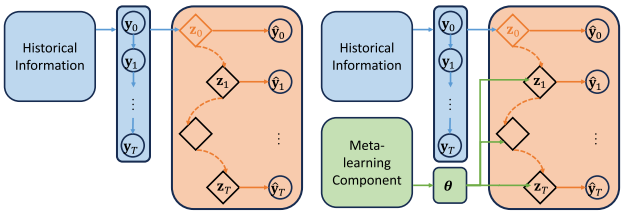
Yunyao Cheng, Kaixuan Chen, Kai Zhao, and Christian S. Jensen are with the Department of Computer Science, Aalborg University, 9220 Aalborg, Denmark (e-mail: yunyaoc@cs.aau.dk; kchen@cs.aau.dk; kaiz@cs.aau.dk; csj@cs.aau.dk).

Chenjuan Guo and Bin Yang are with the Department of Computer Science, East China Normal University, Shanghai 200050, China (e-mail: cjguo@dase.ecnu.edu.cn; byang@dase.ecnu.edu.cn).

Jiandong Xie and Feiteng Huang are with Huawei Cloud Database Innovation Lab, Shenzhen 550029, China (e-mail: xiejiandong@huawei.com; huangfeiteng@huawei.com).

Kai Zheng is with the University of Electronic Science and Technology of China, Chengdu 611731, China (e-mail: zhengkai@uestc.edu.cn).

Digital Object Identifier 10.1109/TKDE.2025.3573673



(a) Sequential LVM (b) Meta-learning LVM

Fig. 2. LVM paradigms.

of an abstract latent state, not directly observed based on historical information, and can derive future observations. To enhance prediction accuracy in few-shot learning, as shown in Fig. 2(b), Jiang et al. [3] propose a meta-learning-based sequential LVM paradigm, where the latent dynamic function $\mathbf{z}_i = f(\mathbf{z}_{i-1}; \boldsymbol{\theta})$ is fed meta-knowledge $\boldsymbol{\theta}$ as additional information. **Patterns such as trends and seasonality in time series, as shown in Fig. 1, which are presented in an explicit and quantifiable form, can be regarded as meta-knowledge.** The meta-knowledge $\boldsymbol{\theta}$ is generated by the meta-learning component that can be a trainable neural network.

Although existing methods for few-shot sequence forecasting achieve significant advancements, two important challenges remain to be addressed.

Challenge 1: It is challenging for existing few-shot learning methods to achieve high forecasting accuracy on time series with **long-term dependencies**. These dependencies refer to patterns such as trend and seasonality that span an entire time series, and they are prevalent in time series. In contrast, existing sequential LVMs [3], [4], [5], [6] focus mainly on image sequence forecasting, where they predict future frames based on the current frame and where short-term dependencies occur due to the continuity of object motion. Thus, LVMs typically rely on short-term modeling, where the current latent state is derived from the previous one. Although attention mechanisms [7], [36] help capture global dependencies, their performance deteriorates in few-shot settings due to high model complexity, which increases the risk of overfitting. Moreover, these methods lack explicit meta-learning capabilities and must relearn attention weights from scratch for each new task using limited data, which hinders generalization. As a result, capturing long-term dependencies remains challenging.

Challenge 2: It is challenging for existing methods to explicitly model and capture **varying meta-knowledge**. As shown in Fig. 1, time series often exhibit varying meta-knowledge, making it difficult for a single dynamic function to capture diverse latent dynamics. The meta-learning component proposed by Jiang et al. [3] is a black-box neural network, resulting in a simple and implicit, rather than complex and explicit, learning of meta-knowledge. This makes it difficult for the model to capture complex patterns. Additionally, explicit meta-knowledge is crucial in many analyses, as it provides users with information on seasonality and other relevant features.

To address these challenges, we propose MetaGP, a meta-learning-based Gaussian process latent variable model. MetaGP tackles the two challenges as follows.

Addressing challenge 1: To enable MetaGP to capture long-term dependencies in time series in few-shot learning, we propose a novel Gaussian process latent variable model based on LVM and meta-learning theories. A Gaussian process [64], [65] models stochastic functions, with its kernel function defining the measure of similarity in the input space. The kernel function can ensure that observations far apart still maintain strong correlations, thereby enabling the capture of long-term dependencies. Hence, we derive and design a Gaussian process as the dynamic function of the LVM, with its kernel function serving as the meta-knowledge for meta-learning, to address the challenge of capturing long-term dependencies in time series in the few-shot learning, which is difficult for sequential LVMs.

Addressing challenge 2: To enable MetaGP to explicitly model and capture varying meta-knowledge, we propose a novel meta-learning component called Kernel Association Search (KAS) by neural architecture search and the kernel association theories [45], [51]. Unlike traditional meta-learning components with black-box meta-knowledge, MetaGP adopts an explicit approach to generate meta-knowledge. The meta-knowledge in MetaGP is the kernel function of the Gaussian process LVM, where kernel association theory ensures that more complex kernel functions can be generated from a simple prior which is a collection of basic kernel functions. Both the prior and the final learned meta-knowledge possess analytic forms, making them explicit. Then, to enhance KAS to handle more complex patterns in multivariate time series, we propose cross-variable weights, which is a learnable parameter matrix used to control the weights of correlations among variables. Furthermore, KAS utilizes neural architecture search theory to provide an end-to-end meta-learning approach for MetaGP, enabling flexible integration with neural networks and Gaussian process LVMs.

In summary, it is crucial to enable LVMs to capture long-term dependencies in few-shot time series forecasting. Moreover, modeling and capturing varying meta-knowledge explicitly is essential to improve prediction accuracy and interpretability. We make three main contributions:

- We propose a novel meta-learning Gaussian process latent variable model (MetaGP) that, in few-shot learning, can capture long-term dependencies in time series.
- We propose KAS, a novel meta-learning component that can model and capture varying meta-knowledge and correlations in multivariate time series explicitly.
- We report on extensive experiments on simulated physical and real-world biological few-shot datasets, finding that MetaGP is capable of state-of-the-art prediction accuracy. We validate the advantages of MetaGP at capturing long-term dependencies in time series. Furthermore, we analyze and visualize the varying meta-knowledge and correlations captured by KAS.

II. Related Work

Few-shot Learning for Time Series: Few-shot learning is commonly applied to classification and forecasting tasks on time series data. Zhong et al. [23] employ meta-learning approaches, integrating time series classification models with domain

knowledge from the geological sector to detect aftershocks. Chen et al. [21] utilize contrastive learning to alleviate the few-shot challenge in high-frequency time series classification. Montero-Manso et al. [20] and Oreshkin et al. [13] report the first attempts to address few-shot learning for univariate time series forecasting. Montero-Manso et al. [20] employ statistical methods to weight and combine forecast results, while Oreshkin et al. [13] combine meta-learning with neural networks, both incorporating strategies that leverage limited samples and temporal domain knowledge. Jiang et al. [3] introduce a sequential latent variable model that uses the support set as a prior, combined with knowledge derived from few-shot support series, to mitigate the few-shot challenge in deriving dynamic functions. In summary, existing approaches to few-shot learning in scientific time series data predominantly adopt Bayesian principles and utilize small amounts of data and time series-related domain knowledge as priors to address this challenge. However, these methods face challenges in handling long-term dependencies and capturing meta-knowledge.

Long-term Dependencies Learning: Existing approaches to long-term dependency learning in time series forecasting rely mainly on attention mechanisms. The Transformer [35] is adopted due to its strength in modeling long-range dependencies via self-attention. LogTrans [14] introduces sparse attention for this purpose. Subsequent methods, such as Informer [30], Autoformer [29], Triformer [36], Pyraformer [15], and FEDformer [77], propose attention variants tailored to temporal dynamics. Transformer-based models have since evolved in two directions: channel-independent approaches (e.g., PatchTST [7] and Pathformer [16]), focusing on intra-channel patterns; and cross-channel models (e.g., Crossformer [17], CARD [18], and iTransformer [19]), focusing on the explicit capture of inter-channel dependencies to enhance long-term forecasting performance. However, these methods face challenges in few-shot learning scenarios. Attention mechanisms tend to overfit when training samples are limited, and they lack meta-learning capabilities that can enable them to generalize effectively under data scarcity.

Gaussian Processes: Applications of Gaussian processes in neural networks occur in two settings—deep Gaussian processes and deep kernel learning. Deep Gaussian processes [64], [65], [66] replace each layer of a neural network with a Gaussian process to overcome overfitting. However, it is difficult to study correlations of time series explicitly with these methods. Next, deep kernel learning combines the structural properties of deep architectures with the non-parametric flexibility of kernel methods. Wilson et al. [53], [54] combine a neural network and a Gaussian process layer with a spectral mixture kernel. Al-Shedivat et al. [44] propose to learn kernels with LSTMs so that the kernels encapsulate the properties of LSTMs. However, these methods lack the ability to model the diversity seen in multivariate time series. In addition, to capture the multivariate covariance among time series, multi-task Gaussian process models [67], [68], [69], [70] utilize fixed and pre-defined kernel functions, which limit their ability to capture diverse correlations.

Kernel Learning: Gaussian process-based kernel learning methods are used for time series forecasting. These methods generate new kernels by combining existing kernels [71] via kernel association. A selection strategy determines how to select the best kernels, using, e.g., greedy search [38], [45], [71], [72], [73]. However, existing kernels learning approaches suffer from high time complexity. These methods utilize the Bayesian information criterion to select optimal kernel functions. Sun et al. [74] propose a neural network that alternates between stacked linear and interactive layers to simulate kernel association. However, the capacity of the kernel function space is fixed, which may lead to sub-optimal kernels. We propose a dynamic and efficient kernel learning framework that is integrated well into a deep architecture.

III. Preliminaries

A. Problem Definition

We consider a multivariate time series $\mathbf{Y} \in \mathbb{R}^{t \times N}$, where t denotes the number of observations and N is the number of time series, or variables. An F-timestamps-ahead time series forecasting model $\mathcal{F}$ takes as input L historical observations $\mathbf{y}_{t-L+1}, \mathbf{y}_{t-L+2}, \ldots, \mathbf{y}_t$, with $L \ll t$, and predicts the F future observations $\mathbf{y}_{t+1}, \mathbf{y}_{t+2}, \ldots, \mathbf{y}_{t+F}$:

$$\mathcal{F}_{\mathbf{\Gamma}} \left(\mathbf{y}_{t-L+1}, \mathbf{y}_{t-L+2}, \ldots, \mathbf{y}_t \right) = \left(\mathbf{y}_{t+1}, \mathbf{y}_{t+2}, \ldots, \mathbf{y}_{t+F} \right), \quad (1)$$

where Γ is a collection of learnable parameters and functions.

B. Gaussian Process

A Gaussian process is a joint Gaussian distribution consisting of a set of random variables [40]. Since training and inference proceed differently, we introduce them separately.

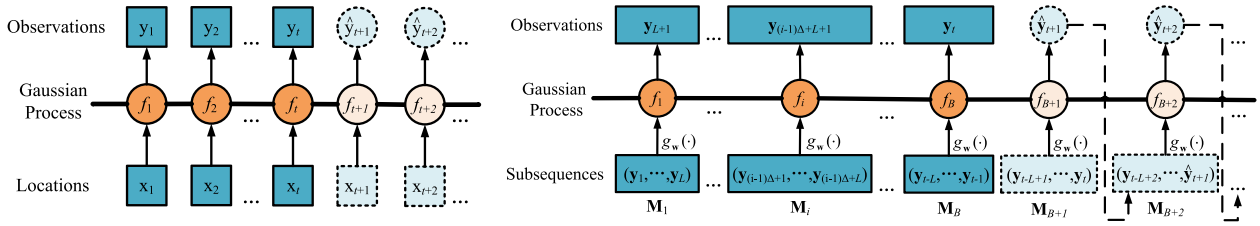
Training: As Fig. 3(a) shows, given t training samples of location and observation pairs $(\mathbf{x}, \mathbf{y}) = \{(\mathrm{x}_i, \mathrm{y}_i) | i = 1, 2, \ldots, t\}$ with additive, independent, and identically distributed Gaussian noise ε, we have $\mathrm{y}_i = f(\mathrm{x}_i) + \varepsilon$. For simplicity, we use $f(\mathrm{x}_i)$ and f_i interchangeably, and we assume that $\mathbf{f} = f(\mathbf{x}) = (f_1, f_2, \ldots, f_i, \ldots, f_t)$ is a collection of random variables. We assume a traditional Gaussian process and use "locations" in place of timestamps. A Gaussian process can be specified by its given priors—its mean $\mu(\mathrm{x}_i)$ and kernel function $k_\theta(\mathrm{x}_i, \mathrm{x}_j)$ that is parameterized by θ. Specifically, θ can be given as prior values or learned through optimization [41]. The kernel function associates any pair of two random variables at locations x_i and x_j. We use i, j to denote any two indices in a subsequence or a kernel. The mean and kernel functions are denoted as:

$$\begin{aligned} \mu(\mathrm{x}_i) &= \mathbb{E}\left[f(\mathrm{x}_i) \right] \\ k_\theta(\mathrm{x}_i, \mathrm{x}_j) &= \mathbb{E}\left[\left(f(\mathrm{x}_i) - \mu(\mathrm{x}_i) \right) \left(f(\mathrm{x}_j) - \mu(\mathrm{x}_j) \right) \right], \end{aligned} \quad (2)$$

giving rise to the Gaussian process:

$$\mathbf{f} \sim \mathcal{GP}\left(\boldsymbol{\mu}(\mathbf{x}), \mathbf{K}(\mathbf{x}, \mathbf{x}) \right), \quad (3)$$

where $\boldsymbol{\mu}(\mathbf{x}) = (\mu(\mathrm{x}_1), \mu(\mathrm{x}_2), \ldots, \mu(\mathrm{x}_t))$ denotes the mean vector containing values for all training observations and $\mathbf{K}(\mathrm{x}, \mathrm{x})$ is the covariance matrix that contains covariance values computed by kernel function $k_\theta(\mathrm{x}_i, \mathrm{x}_j)$ for each pair of training locations x_i and x_j.



(a) Classical Gaussian Process (b) Gaussian Process Latent Variable Model

Fig. 3. Gaussian processes.

Although some studies refer to kernel functions and covariance matrices interchangeably, we distinguish between them. Specifically, we use kernel functions to represent the analytic forms of kernels that define priors on a Gaussian process, and we use covariance matrices to represent the matrices of the covariance values computed by kernel functions.

Inference: Inference aims to predict random variables $\mathbf{f}_* = (f_{t+1}, f_{t+2}, \ldots)$ at test locations $\mathbf{x}_* = (\mathrm{x}_{t+1}, \mathrm{x}_{t+2}, \ldots)$. Assuming noise ε with variance σ^2, we formulate the distribution of the training observations $\mathbf{y}$ and the prediction random variables $\mathbf{f}_*$ under the priors, i.e., kernel functions, as follows.

$$\begin{bmatrix} \mathbf{y} \\ \mathbf{f}_* \end{bmatrix} = \mathcal{N}\left(\begin{bmatrix} \boldsymbol{\mu}(\mathbf{x}) \\ \boldsymbol{\mu}(\mathbf{x}_*) \end{bmatrix}, \begin{bmatrix} \mathbf{K}(\mathbf{x},\mathbf{x}) + \sigma^2\mathbf{I} & \mathbf{K}(\mathbf{x},\mathbf{x}_*) \\ \mathbf{K}(\mathbf{x}_*,\mathbf{x}) & \mathbf{K}(\mathbf{x}_*,\mathbf{x}_*) \end{bmatrix} \right), \quad (4)$$

where $\boldsymbol{\mu}(\mathbf{x})$ and $\boldsymbol{\mu}(\mathbf{x}_*)$ denote the mean vectors and $\mathbf{K}(\mathbf{x},\mathbf{x})$, $\mathbf{K}(\mathbf{x}_*,\mathbf{x})$, $\mathbf{K}(\mathbf{x},\mathbf{x}_*)$, and $\mathbf{K}(\mathbf{x}_*,\mathbf{x}_*)$ are the covariance matrices. Then the distribution of the prediction $\mathbf{f}_*$ has the form:

$$\mathbf{f}_* | \mathbf{x}, \mathbf{y}, \mathbf{x}_* \sim \mathcal{N}\left(\mathbb{E}\left[\mathbf{f}_*\right], \mathrm{Cov}\left[\mathbf{f}_*\right]\right), \quad (5)$$

where

$$\begin{aligned} \mathbb{E}\left[\mathbf{f}_*\right] &= \boldsymbol{\mu}(\mathbf{x}_*) + \mathbf{K}\left(\mathbf{x}_*,\mathbf{x}\right)\left[\mathbf{K}(\mathbf{x},\mathbf{x}) + \sigma^2\mathbf{I}\right]^{-1}\left(\mathbf{y} - \boldsymbol{\mu}(\mathbf{x})\right) \\ \mathrm{Cov}[\mathbf{f}_*] &= \mathbf{K}(\mathbf{x}_*,\mathbf{x}_*) - \mathbf{K}(\mathbf{x}_*,\mathbf{x})\left[\mathbf{K}(\mathbf{x},\mathbf{x}) + \sigma^2\mathbf{I}\right]^{-1}\mathbf{K}(\mathbf{x},\mathbf{x}_*) \end{aligned} \quad (6)$$

When we conduct point estimation, i.e., predict $\mathbf{y}_*$ given $\mathbf{x}_*$, we do not use the random variables $\mathbf{f}_*$ as the prediction values. Instead, we use the expectations $\mathbb{E}[\mathbf{f}_*]$ as the predictions, i.e., $\hat{\mathbf{y}}_* = (\hat{y}_{t+1}, \hat{y}_{t+2}, \ldots) = (\mathbb{E}(f_{t+1}), \mathbb{E}(f_{t+2}), \ldots)$, and the standards deviations $\mathrm{Cov}[\mathbf{f}_*]$ can be used to obtain confidence intervals.

C. Kernel Functions

Kernel functions are the priors used by a Gaussian process and are the analytic forms of the functions that we are modeling. A kernel function is a function of $\mathrm{x}_i - \mathrm{x}_j$ [40]. A kernel function thus specifies the covariance between a pair of random variables using a function of their location distance. For example, the squared exponential (SE) kernel is one of the most commonly used kernel functions in Gaussian processes. The observations y_i and y_j at two nearby locations x_i and x_j are expected to be close in a Gaussian process. In other words, the closer the locations, the more similar the observations should be. Thus, the SE kernel function represents an interpretable pattern that captures short-term dependencies.

TABLE I
BASIC KERNEL FUNCTIONS

Kernel Function	Interpretable Pattern	Analytic Form	Parameter(s)
SE	Short-term	$\sigma_k^2 \exp\left(-\frac{\|\|\mathrm{x}_i - \mathrm{x}_j\|\|}{2l^2}\right)$	l
PER	Seasonality	$\sigma_k^2 \exp\left(-\frac{2\sin^2(\frac{\pi\|\mathrm{x}_i - \mathrm{x}_j\|)}{p}}{l^2}\right)$	l,p
LIN	Trend	$\sigma_k^2(\mathrm{x}_i - c)(\mathrm{x}_j - c)$	c
RQ	Long-term	$\sigma_k^2\left(1 + \frac{\|\|\mathrm{x}_i - \mathrm{x}_j\|\|}{2al^2}\right)^{-a}$	l,$a > 0$

The SE kernel is defined as:

$$k_\theta(\mathrm{x}_i, \mathrm{x}_j) = \sigma_k^2 \exp\left(-\frac{||\mathrm{x}_i - \mathrm{x}_j||}{2l^2}\right), \quad (7)$$

where $\theta = \{\sigma_k, l\}$ denotes the parameters. Specifically, σ_k denotes the scale factor of the covariance and l represents the parameter length-scale. A shorter length-scale indicates that observations $\mathbf{y}$ vary more rapidly across locations $\mathbf{x}$.

Numerous kernel functions exist beyond the SE kernel function that describe observations in an interpretable way. Table I lists basic kernel functions and the patterns they target. The periodic analytic form of the PER kernel captures periodic correlations, while the linear analytic form of the LIN kernel captures linear correlations. The RQ kernel reduces to the SE kernel when its parameter $a \to \infty$ [42]. The covariance captured via the RQ kernel decays more slowly with increasing distance than in the case of the SE kernel, as SE is the inverse of an exponential function and RQ is a power function with power $-a$. Therefore, the RQ kernel can capture long-term dependencies. Another study [43] provides a detailed explanation of the mathematical properties of different kernel functions, thus serving as a comprehensive kernel guide.

IV. METHODOLOGY

A. MetaGP Framework

Fig. 4 illustrates the overall framework of MetaGP, which covers two main stages: training and inference. In the training stage, the model learns a Gaussian process-based dynamic function by processing historical time series segments. These segments are used to predict the next-step observations, enabling the model to capture temporal patterns and dependencies.

The learned model consists of three core components. First, an attention-based module learns the temporal structure and

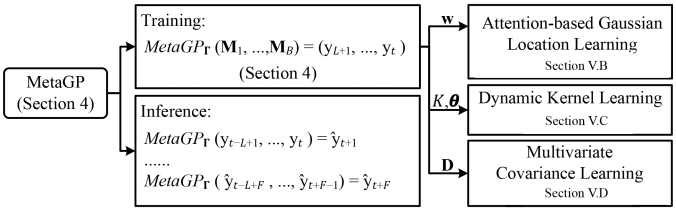


Fig. 4. MetaGP framework encompassing training and inference.

highlights important time steps. Second, a dynamic kernel learning module builds flexible kernel functions to model complex dependencies. Third, a multivariate covariance learning component captures correlations across different variables in the time series.

In the inference stage, the model generates future predictions iteratively. It takes recent observations (or previously predicted values) as input and forecasts the next time step. This step-by-step process allows the model to incorporate newly predicted information adaptively and to produce a sequence of future values. Together, these modules aim to enable MetaGP to perform accurate and interpretable few-shot time series forecasting, even in complex and data-scarce scenarios.

Traditional Gaussian process models use timestamps as the Gaussian locations for regression tasks. In other words, they predict observations given their locations or timestamps. In contrast, Gaussian process LVMs generate latent variables sequentially using historical observations in time order. We define a Gaussian process LVM for multivariate time series forecasting that recursively predicts each future observation given a subsequence of historical observations. Further, the latent variables in the full time series are modeled using Gaussian process kernel functions. More specifically, this process contains two procedures—training and inference.

Training: In order to establish a Gaussian process dynamic function of latent variables as exemplified in Fig. 3(b), the multivariate time series $\mathbf{Y} \in \mathbb{R}^{t\times N}$ is sliced into a series $\mathbf{M}$ of B subsequences using moving a time window of length L and step size Δ: $\mathbf{M} = (\mathbf{M}_1, \mathbf{M}_2, \ldots, \mathbf{M}_i, \ldots, \mathbf{M}_B)$, where $\mathbf{M}_i \in \mathbb{R}^{L\times N}$ and $B = (t - L + 1)/\Delta$. The i-th subsequence $\mathbf{M}_i$ is denoted as:

$$\mathbf{M}_i = \left(\mathbf{y}_{(i-1)\Delta+1}, \mathbf{y}_{(i-1)\Delta+2}, \ldots, \mathbf{y}_{(i-1)\Delta+L}\right) \tag{8}$$

As in traditional Gaussian process training, we define the expectation of a future observation as $\mathbf{y}_{(i-1)\Delta+L+1} = \mathbb{E}(f_i)$, where $f_i = f(g_{\mathbf{w}}(\mathbf{M}_i))$ and $g_{\mathbf{w}}(\cdot)$ is a neural network used to encode the historical observations, as shown in Fig. 3(b). To enable time series forecasting, we build a temporal correlation between a historical subsequence $\mathbf{M}_i$ and a predicted observation $\mathbf{y}_{(i-1)\Delta+L+1}$. We then match $g_{\mathbf{w}}(\mathbf{M}_i)$ and $\mathbf{y}_{(i-1)\Delta+L+1}$ with x_i and y_i in the training stages as shown in Section III-B, and then we formulate the dynamic function as follows.

$$\begin{aligned}(f_1, \ldots, f_i, \ldots, f_B) \sim \mathcal{GP}\left(\boldsymbol{\mu}\left(g_{\mathbf{w}}(\mathbf{M})\right),\right.\\ \left.\mathbf{K}\left(g_{\mathbf{w}}(\mathbf{M}), g_{\mathbf{w}}(\mathbf{M})\right)\right),\end{aligned} \tag{9}$$

where $\boldsymbol{\mu}(g_{\mathbf{w}}(\mathbf{M})) = (\mu(g_{\mathbf{w}}(\mathbf{M}_1)), \ldots, \mu(g_{\mathbf{w}}(\mathbf{M}_B)))$ is the mean vector and $\mathbf{K}(g_{\mathbf{w}}(\mathbf{M}), g_{\mathbf{w}}(\mathbf{M}))$ is the covariance matrix.

To achieve better forecasting accuracy, our goal is to learn the MetaGP model which contains three parts. The first is a neural network $g_{\mathbf{w}}(\cdot)$ that encodes the subsequences $\mathbf{M}$ as latent variables. The second is a final kernel function $K_{\boldsymbol{\theta}}$ that is an association of kernel functions guided by a learnable cross-variable weight matrix $\mathbf{D}$, which is the third part. Here $g_{\mathbf{w}}(\cdot)$ is parameterized by $\mathbf{w}$, and $K_{\boldsymbol{\theta}}$ is parameterized by $\boldsymbol{\theta}$. Thus, the model has parameters $\boldsymbol{\Gamma} = \{\mathbf{w}, K, \boldsymbol{\theta}, \mathbf{D}\}$ and can be stated as follows.

$$MetaGP_{\boldsymbol{\Gamma}}\left(\mathbf{M}_1, \ldots, \mathbf{M}_B\right) = \left(\mathbf{y}_{L+1}, \ldots, \mathbf{y}_t\right) \tag{10}$$

We train the model using $\mathbf{M}_1, \ldots, \mathbf{M}_B$, and $\mathbf{y}_{L+1}, \ldots, \mathbf{y}_t$, in order to learn $\mathbf{w}$, K, $\boldsymbol{\theta}$, and $\mathbf{D}$.

Inference: To iteratively infer F future observations, as illustrated in Section III-B, we use Gaussian process inference. Specifically, in the first iteration, we match $g_{\mathbf{w}}(\mathbf{M}_{B+1})$, where $\mathbf{M}_{B+1} = \mathbf{y}_{t-L+1}, \ldots, \mathbf{y}_t$, and $\hat{\mathbf{y}}_{t+1}$ with x_{t+1} and $\hat{y}_{t+1}$, and thus infer $\mathbf{y}_{t+1}$ using (4)–(6), as shown in Fig. 3(b). Unlike Gaussian process inference, which takes arbitrary future locations as input and predicts their observations, Gaussian process LVMs predict iteratively, as follows.

$$\begin{aligned}MetaGP_{\boldsymbol{\Gamma}}(\mathbf{y}_{t-L+1}, \ldots, \mathbf{y}_t) &= \hat{\mathbf{y}}_{t+1}\\ MetaGP_{\boldsymbol{\Gamma}}(\mathbf{y}_{t-L+2}, \ldots, \hat{\mathbf{y}}_{t+1}) &= \hat{\mathbf{y}}_{t+2}\\ &\ldots\\ MetaGP_{\boldsymbol{\Gamma}}(\hat{\mathbf{y}}_{t-L+F}, \ldots, \hat{\mathbf{y}}_{t+F-1}) &= \hat{\mathbf{y}}_{t+F},\end{aligned} \tag{11}$$

where the sequence $\hat{\mathbf{y}}_{t+1}, \hat{\mathbf{y}}_{t+2}, \ldots, \hat{\mathbf{y}}_{t+F}$ denotes F predictions.

B. Attention-Based Gaussian Location Learning

To capture and highlight salient temporal features in each subsequence, we propose an attention-based Gaussian location learning mechanism. Unlike traditional methods that treat all time steps equally, the attention mechanism dynamically assigns different weights to the input, enabling selective extraction of informative temporal patterns. This enhances the quality of latent representations with the goal of improving the accuracy and interpretability of MetaGP.

The training of the MetaGP framework is illustrated in Fig. 5. The framework takes as input subsequences $\mathbf{M} = (\mathbf{M}_1, \mathbf{M}_2, \ldots, \mathbf{M}_i, \ldots, \mathbf{M}_B)$, where $\mathbf{M}_i \in \mathbb{R}^{L\times N}$, and outputs predictions $\mathbf{y}_{L+1}, \ldots, \mathbf{y}_{L+i}, \ldots, \mathbf{y}_{L+j}, \ldots, \mathbf{y}_t$ for training.

The first component, attention-based Gaussian location learning, takes $\mathbf{M}$ as input and outputs latent variables $\mathbf{H} = (\mathbf{h}_1, \ldots, \mathbf{h}_i, \ldots, \mathbf{h}_j, \ldots, \mathbf{h}_B)$, where $\mathbf{h}_i \in \mathbb{R}^N$ represents the location of a Gaussian process; see Fig. 5(a). We use patch-attention [36] to capture the temporal dynamics in subsequence $\mathbf{M}_i$ and encode subsequences $\mathbf{M}$ to Gaussian locations $\mathbf{H}$. MetaGP utilizes attention structures to capture local features in subsequence $\mathbf{M}_i$ and utilizes the Gaussian process to construct global features among all subsequences in $\mathbf{M}$. This enables MetaGP to capture multiscale information from time series, and short-term and long-term dependencies.

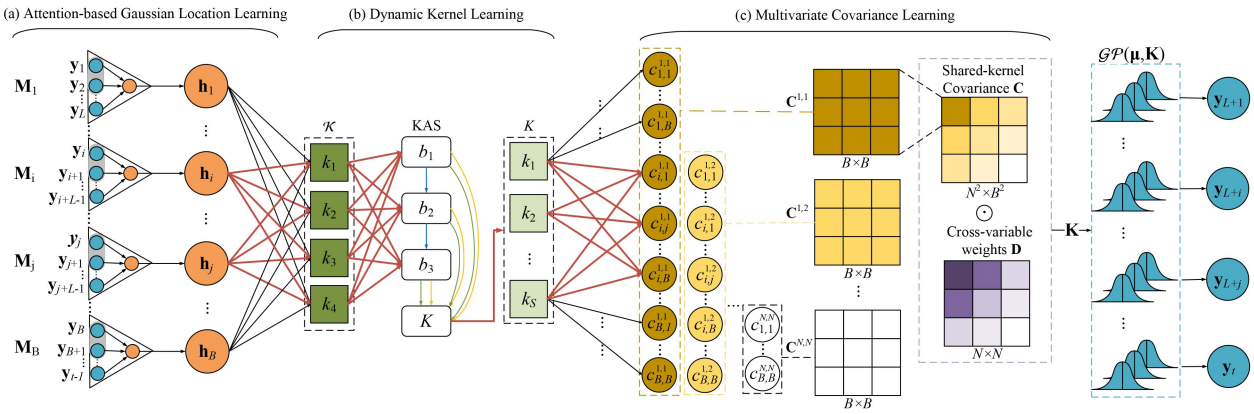


Fig. 5. MetaGP training.

The mechanism to achieve the attention-based Gaussian locations $\mathbf{H}$ is defined as follows.

$$\begin{aligned}\mathbf{H} &= (\mathbf{h}_1, \ldots, \mathbf{h}_i, \ldots, \mathbf{h}_j, \ldots, \mathbf{h}_B) \\ &= (g_{\mathbf{w}}(\mathbf{M_1}), \ldots, g_{\mathbf{w}}(\mathbf{M_i}), \ldots, g_{\mathbf{w}}(\mathbf{M_j}), \ldots, g_{\mathbf{w}}(\mathbf{M_B}))\end{aligned} \tag{12}$$

Next, representations $\mathbf{H}$ are input into the Gaussian layer, which is formulated as follows.

$$(f_1, \ldots, f_i, \ldots, f_j, \ldots, f_B) \sim \mathcal{GP}\left(\boldsymbol{\mu}(\mathbf{H}), \mathbf{K}(\mathbf{H}, \mathbf{H})\right), \tag{13}$$

where $\boldsymbol{\mu}(\mathbf{H}) = (\mu(\mathbf{h_1}), \ldots, \mu(\mathbf{h_i}), \ldots, \mu(\mathbf{h_j}), \ldots, \mu(\mathbf{h_B}))$ and $\mathbf{K}(\mathbf{H}, \mathbf{H})$ are the mean vector and the covariance matrix.

C. Dynamic Kernel Learning

Dynamic kernel learning aims at dynamically learning and searching the associations of kernels to consider the different meta-knowledge patterns in multivariate time series. The dynamic kernel learning starts from a basic kernel set $\mathcal{K}$. In Fig. 5(b), we assume $\mathcal{K} = \{k_1, \ldots, k_4\}$, corresponding to the four kernel functions shown in Table I. The component automatically expands and learns potential new kernels via kernel association search (KAS) to achieve the final kernel function K_θ, which is an association of basic kernels, e.g., k_1 and k_2, and expanded kernels, e.g., k_S. For simplicity, we denote K_θ as K.

Instead of using a simple and fixed kernel function to model a fixed meta-knowledge pattern among subsequences $\mathbf{M}$, which fails to capture the diversity among time series, such as combinations of growing trends and periodic patterns, we propose to learn a kernel function K to contend with such diversity. It encompasses *kernel association*, which defines the rules of structural exploration of new kernels, and *kernel search*, which applies two kernel search methods to obtain the final kernel K.

1) Kernel Association: A more complex kernel function can be obtained by the addition and multiplication of existing kernel functions because positive semi-definite covariance matrices are closed under addition and multiplication [45].

Lemma 4.1: Let k_1 and k_2 be kernels and let $+$ and $\times$ denote addition and multiplication, respectively. Then $k_1 + k_2$ and $k_1 \times k_2$ are kernels.

Algorithm 1: Greedy Kernel Search.

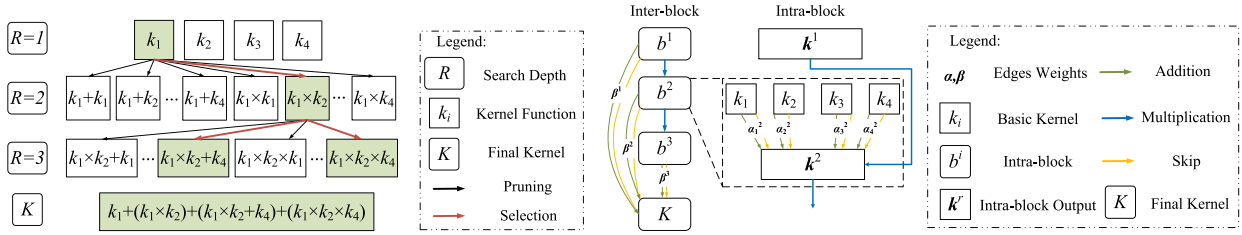
Input: Kernel set $\mathcal{K} = \{k_1, k_2, \ldots, k_{|\mathcal{K}|}\}$, operations $\mathcal{O} = \{+, \times\}$, depth R
Output: Final kernel K
1: Initialize K by randomly selecting $k \in \mathcal{K}$
2: **for** $r = 1, \ldots, R$ **do**
3: Generate candidate kernels using $\mathcal{O}$ and $\mathcal{K}$
4: Select the kernel minimizing the forecasting loss and update K
5: **end for**
6: **return** K

Lemma 4.1 allows us to expand basic kernels into high order kernels with power larger than 2. For example, LIN is a basic kernel, and LIN^2 is a second-order kernel. The basic kernel LIN denotes a trend pattern, while the high order kernel LIN^2 denotes a quadratic pattern. When a high order kernel is combined with basic kernels into a final kernel K, it exhibits a more complex analytic form than the basic kernels. This procedure is called kernel association [46]. The intuition is that kernels with different orders should be included in K because basic kernels capture general patterns of time series, while higher order kernels capture higher order information. As more higher order kernels are included in K, K approximates a real-world pattern better. This is similar to polynomial approximation [47].

Specifically, we define a basic kernel set $\mathcal{K} = \{k_1, k_2, \ldots, k_{|\mathcal{K}|}\}$, where k_i is a kernel, and $|\mathcal{K}|$ is the number of kernels in $\mathcal{K}$. Then we define the operation set $\mathcal{O} = \{+, \times\}$. Our goal is to discover a suitable final kernel K, which is an expanded high order kernel obtained via operations in $\mathcal{O}$.

2) Kernel Search: We procee to introduce two kernel search methods—greedy kernel search, used by existing models [38], [45], [48], [49], and KAS, a novel search method.

Greedy Kernel Search: As shown in Fig. 6(a), assuming a basic kernel set $\mathcal{K} = \{k_1, k_2, k_3, k_4\}$, an operation set $\mathcal{O} = \{+, \times\}$, and a search depth $R = 3$, the goal of greedy kernel search is to discover a final kernel K. The detailed procedure is presented in Algorithm 1.



(a) Greedy Kernel Search. (b) Kernel Association Search.

Fig. 6. Kernel search methods.

The method starts by randomly selecting a basic kernel from kernel set $\mathcal{K}$, forming an initial kernel K. It then evaluates each candidate kernel formed by combining K with other kernels using operations from $\mathcal{O} = \{+, \times\}$, selecting the combination that minimizes the forecasting loss $\mathcal{L}_{que}$. This procedure continues until reaching a predefined search depth R. In each iteration, new candidate kernels are generated by combining previously selected kernels with basic kernels, continually optimizing K. The final kernel K is the sum of all selected kernels from each iteration.

While greedy kernel search discovers and expands kernels automatically, forecasting loss computation must be performed each time a candidate kernel is checked, causing a high time complexity of $O(|\mathcal{K}|^R)$.

Kernel Association Search: To reduce the complexity, we propose a novel 2-level kernel search method (KAS). Inspired by neural architecture search methods [50], [51], it includes intra-block and inter-block kernel associations, as shown in Fig. 6(b). Each intra-block association b^r constructs an r-th order kernel $\boldsymbol{k}^r$, and the inter-block association sums kernels $\boldsymbol{k}^1, \ldots, \boldsymbol{k}^R$ to obtain the final kernel K. The algorithm detailing the procedure is presented in Algorithm 2.

Each intra-block association b^r encompasses two steps: addition and multiplication. The addition step takes basic kernels $\mathcal{K} = \{k_1, k_2, \ldots, k_{|\mathcal{K}|}\}$ as input, producing an intermediate kernel $\boldsymbol{k}^r$ as a weighted sum based on edge operations $\mathcal{E} = \{+, skip\}$. The weight $\alpha^r_{i,j}$ determines whether kernel k_i is included $(+)$ or omitted $(skip)$ when constructing $\boldsymbol{k}^r$. Formally, the intermediate kernel is calculated as follows.

$$\boldsymbol{k}^r = \sum_{i=1}^{|\mathcal{K}|} \sum_{j=1}^{|\mathcal{E}|} \frac{\exp(\alpha^r_{i,j})}{\sum_{j=1}^{|\mathcal{E}|} \exp(\alpha^r_{i,j})} e_j(k_i), \tag{14}$$

where e_j is the j-th operation in $\mathcal{E}$.

The multiplication step updates the intermediate kernel to a higher-order kernel by multiplying it with the previous kernel: $\boldsymbol{k}^r \leftarrow \boldsymbol{k}^{r-1} \times \boldsymbol{k}^r$, with $\boldsymbol{k}^0 = 1$.

Inter-block associations aggregate kernels from multiple intra-blocks. Each intra-block kernel $\boldsymbol{k}^r$ is integrated into the final kernel K through weighted edges β^r_j, deciding inclusion or omission similarly and using operations in $\mathcal{E}$:

$$K = \sum_{r=1}^{R} \sum_{j=1}^{|\mathcal{E}|} \frac{\exp(\beta^r_j)}{\sum_{j=1}^{|\mathcal{E}|} \exp(\beta^r_j)} e_j(\boldsymbol{k}^r) \tag{15}$$

Algorithm 2: Kernel Association Search (KAS).

Input: Basic kernel set $\mathcal{K} = \{k_1, k_2, \ldots, k_{|\mathcal{K}|}\}$, edges $\mathcal{E} = \{+, skip\}$, intra-block number R
Output: Final kernel K

Initialization: Set $\boldsymbol{k}^0 = 1$
for $r = 1, 2, \ldots, R$ **do**
  Intra-block addition:
  for $k_i \in \mathcal{K}$ **do**
    Compute edge weights $\boldsymbol{\alpha}^r_i = (\alpha^r_{i,1}, \ldots, \alpha^r_{i,|\mathcal{E}|})$
    Select operation $e_j \in \mathcal{E}$ s.t. $j = \arg\max_j \alpha^r_{i,j}$
  end for
  $\boldsymbol{k}^r = \sum_{i=1}^{|\mathcal{K}|} \sum_{j=1}^{|\mathcal{E}|} \frac{\exp(\alpha^r_{i,j})}{\sum_j \exp(\alpha^r_{i,j})} e_j(k_i)$
  Intra-block multiplication:
  Update high-order kernel: $\boldsymbol{k}^r \leftarrow \boldsymbol{k}^{r-1} \times \boldsymbol{k}^r$
end for
Inter-block association:
for $r = 1, 2, \ldots, R$ **do**
  Compute edge weights $\boldsymbol{\beta}^r = (\beta^r_1, \ldots, \beta^r_{|\mathcal{E}|})$
  Select operation $e_j \in \mathcal{E}$ s.t. $j = \arg\max_j \beta^r_j$
end for
Construct final kernel:
  $K = \sum_{r=1}^{R} \sum_{j=1}^{|\mathcal{E}|} \frac{\exp(\beta^r_j)}{\sum_j \exp(\beta^r_j)} e_j(\boldsymbol{k}^r)$
Kernel Weighting:
Retrain the model and learn kernel weights $\boldsymbol{\zeta} = (\zeta_1, \ldots, \zeta_S)$
Update final kernel with weights: $K = \sum_{s=1}^{S} \zeta_s k_s$
return K

Finally, kernel weights $\boldsymbol{\zeta} = (\zeta_1, \ldots, \zeta_S)$ are learned by retraining the model to reflect the importance of each kernel:

$$K = \sum_{s=1}^{S} \zeta_s k_s, \tag{16}$$

where ζ_s is the importance of kernel k_s.

The kernel selection is completed in one iteration without traversing the search tree as done in the greedy strategy. The time complexity is $O(|\mathcal{K}|R)$. KAS involves only one instance of forecasting query set loss computation, thereby reducing the time complexity.

D. Multivariate Covariance Learning

Fig. 5(c) shows the third component in the framework, multivariate covariance learning, that aims to capture multivariate

correlations among variables. Taking the Gaussian locations $\mathbf{H}$ and the learned kernel K as input, the component calculates the multivariate covariance matrix $\mathbf{K}(\mathbf{H},\mathbf{H})$ among Gaussian locations $\mathbf{H}$ via the learned kernel K. For simplicity, we denote $\mathbf{K}(\mathbf{H},\mathbf{H})$ by $\mathbf{K}$. We use $\mathbf{K}$ to capture the correlations between each two subsequences of different variables.

To learn $\mathbf{K}$, we first calculate a shared-kernel covariance $\mathbf{C}(\mathbf{H},\mathbf{H})$, denoted by $\mathbf{C}$, via a multi-task Gaussian process [52], a Gaussian process that handles multiple variables; and then we learn a matrix $\mathbf{D}$ that captures the cross-variable weights between each pair of variables. Next, it learns the cross-variable weight matrix $\mathbf{D} \in \mathbb{R}^{N^2}$ that captures the importance between each pair of variables. Finally, we get the multivariate covariance $\mathbf{K}$ as the Hadamard product of $\mathbf{C}$ and $\mathbf{D}$ for the correlations between any two subsequences of any two variables.

1) Shared-Kernel Covariance: The shared-kernel covariance matrix $\mathbf{C} \in \mathbb{R}^{B^2 \times N^2}$ is learned from the attention-based Gaussian locations $\mathbf{H}$ and the kernel function K. An element $\mathbf{C}^{m,n} \in \mathbb{R}^{B^2}$ in $\mathbf{C}$ denotes the covariance between the m-th and n-th variables in $\mathbf{H}$:

$$\mathbf{C} = \begin{bmatrix} \mathbf{C}^{1,1} & \mathbf{C}^{1,2} & \cdots & \mathbf{C}^{1,N} \\ \mathbf{C}^{2,1} & \mathbf{C}^{2,2} & \cdots & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{C}^{N,1} & \mathbf{C}^{N,2} & \cdots & \mathbf{C}^{N,N} \end{bmatrix} \tag{17}$$

Element $\mathbf{C}^{m,n}$ is also a matrix, an element $\mathbf{C}^{m,n}_{i,j}$ of which denotes the temporal covariance between the i-th subsequence of the m-th variable $\mathbf{h}^m_i$ and the j-th subsequence of the n-th variable $\mathbf{h}^n_j$. Assuming that the final kernel K contains S kernel functions, we have:

$$\mathbf{C}^{m,n}_{i,j} = K\left(\mathbf{h}^m_i, \mathbf{h}^n_j\right) = \sum_{s=1}^{S} \zeta_s k_s\left(\mathbf{h}^m_i, \mathbf{h}^n_j\right) \tag{18}$$

2) Cross-Variable Weights: The multi-task Gaussian process captures the multivariate covariance by means of a single shared kernel function K. As this limits the ability to capture diverse correlations between variables, we propose a learnable matrix $\mathbf{D} \in \mathbb{R}^{N^2}$ that represents the cross-variable weights, thus extending the capabilities of a single shared kernel. To ensure that covariance matrix $\mathbf{D}$ is positive semi-definite, we define $\mathbf{D}$ as follows.

$$\mathbf{D} = \mathbf{D}'(\mathbf{D}')^\top + \mathrm{diag}(\boldsymbol{\lambda}), \tag{19}$$

where $\mathbf{D}' \in \mathbb{R}^{N \times V}$ is a low-rank matrix, V $(V < N)$ represents the number of non-zero columns in $\mathbf{C}$, and $\boldsymbol{\lambda}$ is a non-negative vector.

The multivariate covariance matrix $\mathbf{K}$ is calculated using the Hadamard product of the shared-kernel covariance and the cross-variable weights:

$$\mathbf{K} = \sum_{s=1}^{S} \zeta_s \begin{bmatrix} d_{1,1}k_s(\mathbf{h}^1,\mathbf{h}^1) & \cdots & d_{1,N}k_s(\mathbf{h}^1,\mathbf{h}^N) \\ d_{2,1}k_s(\mathbf{h}^2,\mathbf{h}^1) & \cdots & \vdots \\ \vdots & \ddots & \vdots \\ d_{N,1}k_s(\mathbf{h}^N,\mathbf{h}^1) & \cdots & d_{N,N}k_s(\mathbf{h}^N,\mathbf{h}^N) \end{bmatrix}$$

$$= \mathbf{D} \odot \mathbf{C}, \tag{20}$$

where $d_{m,n}$ denotes the weight of $\mathbf{C}^{m,n}$ that constructs the cross-variable correlation between any pair of variables. Thus, the correlation between any two subsequences of two variables also considers the distinct correlation between the two variables. Matrix $\mathbf{K}$ will be fed into the multi-task Gaussian process for training and inference.

E. MetaGP Optimization

Next, we consider parameter optimization in the MetaGP framework. As the multivariate time series forecasting is formulated as a Gaussian process, we use negative log marginal likelihood [53] rather than the general regression loss function from deep learning to optimize parameters.

The negative log marginal likelihood loss function is defined as follows.

$$\mathcal{L} = -[\mathbf{y}^\top(\mathbf{K}+\sigma^2 I)^{-1}\mathbf{y} + \log\det|\mathbf{K}+\sigma^2 I|] + const, \tag{21}$$

where $\mathbf{y}$ denotes a series of observations, $\mathbf{K}$ represents a learned covariance matrix with analytic form K, and σ^2 is the variance of noise. Next, $\mathbf{y}^\top(\mathbf{K}+\sigma^2 I)^{-1}\mathbf{y}$ is a positive semi-definite covariance matrix, $\det|\cdot|$ is the determinant of the argument matrix, and $const$ is a constant term.

The framework learns the analytic form of kernel function K achieved via KAS and parameters $\boldsymbol{\Gamma} = \{\mathbf{w}, K, \boldsymbol{\theta}, \mathbf{D}\}$, where $\mathbf{w}$ denotes the learnable weights in the patch-attention, $\boldsymbol{\theta}$ represents the parameters of kernel K, which includes $\boldsymbol{\alpha}$, $\boldsymbol{\beta}$, $\boldsymbol{\zeta}$, and other parameters related to kernels, and $\mathbf{D}$ is the cross-variable weights. Specifically, the cross-variable weights $\mathbf{D}$ is a positive semi-definite matrix. As kernel computation is closed under the Hadamard product operation, the optimization of $\mathbf{D}$ and $\boldsymbol{\theta}$ represents the parameter optimization of kernel K. Therefore, $\mathbf{D}$ is regarded as a covariance matrix, and the optimization procedures of $\mathbf{D}$ and $\boldsymbol{\theta}$ are combined. We let $\boldsymbol{\Theta} = \{\boldsymbol{\theta}, \mathbf{D}\}$. The framework then learns parameters $\boldsymbol{\Theta}$ and $\mathbf{w}$ jointly via back-propagation using loss function $\mathcal{L}$ and gradient descent. The partial derivative $\frac{\partial \mathcal{L}}{\partial \boldsymbol{\Theta}}$ is defined as follows.

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\Theta}} = \frac{1}{2}\mathrm{tr}\left(\left[\mathbf{K}^{-1}\mathbf{y}\mathbf{y}^\top\mathbf{K}^{-1} - \mathbf{K}^{-1}\right]\frac{\partial \mathbf{K}}{\partial \boldsymbol{\Theta}}\right) \tag{22}$$

The other partial derivative $\frac{\partial \mathcal{L}}{\partial \mathbf{w}}$ is defined as follows.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{1}{2}\sum_{m,n}\sum_{i,j}\Big(\mathbf{K}\left(\mathbf{h}^m_i,\mathbf{h}^n_j\right)^{-1}\mathbf{y}\mathbf{y}^\top\mathbf{K}(\mathbf{h}^m_i,\mathbf{h}^n_j)^{-1} - \mathbf{K}\left(\mathbf{h}^m_i,\mathbf{h}^n_j\right)^{-1}\Big)$$
$$\left\{\left(\frac{\partial \mathbf{K}(\mathbf{h}^m_i,\mathbf{h}^n_j)}{\partial \mathbf{h}^m_i}\right)^\top\frac{\partial \mathbf{h}^m_i}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{K}(\mathbf{h}^m_i,\mathbf{h}^n_j)}{\partial \mathbf{h}^n_j}\right)^\top\frac{\partial \mathbf{h}^n_j}{\partial \mathbf{w}}\right\} \tag{23}$$

Then we update the parameters using gradient descent as follows.

$$\boldsymbol{\Theta} = \boldsymbol{\Theta} - \eta\frac{\partial \mathcal{L}}{\partial \boldsymbol{\Theta}}, \quad \mathbf{w} = \mathbf{w} - \eta\frac{\partial \mathcal{L}}{\partial \mathbf{w}}, \tag{24}$$

TABLE II
DATASET STATISTICS

Dataset	Tasks	Samples	N	Length	Ratio	Shot	L	F
Gravity-16	16	3000	1024	100	5:1:2	15	8	12
Bouncing Ball	4	3000	1024	100	3:0:1	15	8	12
Pendulum	4	3000	1024	100	3:0:1	15	8	12
Mass-spring	4	3000	1024	100	3:0:1	15	8	12
MC	24	177	951	196	5:1:2	15	8	12

where η is a learning rate that is used to control the learning speed.

Given a time series of length T with N variables, the time and space complexity of attention-based Gaussian location learning are $O(NT)$. Dynamic kernel learning has time complexity $O(|\mathcal{K}|RNT)$ and space complexity $O(NT)$. Leveraging KISS-GP [78], multivariate covariance learning reduces both complexities to $O(NT)$. Thus, the overall time and space complexity of MetaGP are $O(|\mathcal{K}|RNT)$ and $O(NT)$, respectively.

V. EXPERIMENTAL STUDY

A. Experimental Setup

1) Datasets and Evaluation Metrics: We perform experimental studies on seven benchmark datasets.

Gravity-16[1]: The Gravity-16 dataset [3] consists of a sequence of image frames capturing the trajectory of a ball under varying initial positions and velocities across 16 different directions of gravity. These continuous images form time-series data.

Mixed-physics[2]: The Mixed-physics dataset comprises three sub-datasets: Bouncing Ball [5], Pendulum, and Mass-spring [4]. The Bouncing Ball dataset describes sequences of image frames capturing the trajectory of a ball under 4 different directions of gravity. Pendulum and Mass-spring are also image sequence datasets, now capturing the dynamics of pendulums and mass springs under 4 different friction coefficients.

MC[3]: The MC dataset, contributed by the Department of Chemistry and Bioscience at Aalborg University, describes a time series of microbial community scales in activated sludge sampled from different regions.

Table II provides statistics of the benchmark datasets, where "Tasks" refers to the total number of tasks utilized for meta-training, meta-validation, and meta-testing, with each task consisting of a dataset formed by a group of time series with similar distributions, "Samples" refers to the average number of time series for all tasks, N denotes the average number of variables, "Length" represents the average number of observations, "Ratio" indicates the ratio of meta training-validation-testing tasks for the entire dataset, "shot" indicates the number of samples used for few-shot learning, and L and F denote the lengths of the historical and forecasting horizons, respectively.

On the Gravity-16 and Mixed-physics datasets, MetaGP aligns the dataset statistics strictly with those reported by Jiang et al. [3]. Specifically, in Gravity-16, the "Ratio" 5:1:2 indicates that MetaGP randomly selects 10 tasks for meta-training, 2 tasks for meta-validation, and 4 tasks for meta-testing. In meta-learning, for each task, the support set consists of "shot" randomly selected samples, and the remaining samples are partitioned into the query set. For the MC, we align the "Ratio", "Shot", L, and F with those of Gravity-16.

Metrics: We evaluate prediction accuracy using four metrics [3]: distance (Dist), valid prediction time distance (VPT-Dist), mean absolute error (MAE), and mean squared error (MSE). In image sequences, MAE and MSE are challenging to use for measuring the error between prediction and ground truth, as the pixel-level moving objects are very small. Therefore, Jiang et al. [3] propose the metrics Dist and VPT-Dist. Specifically, Dist quantifies the distance between the predicted object trajectory and the ground truth trajectory, while VPT-Dist assesses the duration during which the predicted object's trajectory aligns closely with the ground truth trajectory. Additionally, we adopt the commonly used metrics MAE and MSE [7] for the biological dataset MC.

2) Baselines: We consider 6 categories of baselines. *System state as latent variable* [8], [9] do not consider parameters as priors that provide additional information to the model; these include VRNN and DKF. (1) VRNN [8] is a traditional sequential latent variable model that incorporates latent random variables into the hidden state of a recurrent neural network. (2) DKF [9] is a Gaussian-based state space model that introduces a unified algorithm to learn a broad class of linear and non-linear state space models. *System parameter as latent variable* [3], [4], [5], [6] consider parameters as priors; these include DVBF, KVAE, GRU-res, NODE, and RGN-res. (3) DVBF [6] is a method for unsupervised learning and identification of latent Markovian state space models, which treats system parameters as latent variables. (4) KVAE [5], a Kalman variational auto-encoder, incorporates system parameters into the process of sequential latent variable inference. (5) GRU-res [3] is a sequential LVM that incorporates external parameters as priors and employs residual gated recurrent units as the latent dynamic function. (6) NODE [3] is similar to GRU-res, but utilizes the neural ordinary differential equation as the latent dynamic function. (7) RGN-res [4] is similar to GRU-res, but utilizes the residual recurrent generative networks as the latent dynamic function. *Meta-learning LVMs* [3] integrate meta-learning theories into sequential latent variable models; these include meta-GRU-res, meta-NODE, and meta-RGN-res. (8) Meta-GRU-res is the meta-learning version of GRU-res, which shows advantages at few-shot learning on image sequences. (9) Meta-NODE is the meta-learning version of NODE. (10) Meta-RGN-res is the meta-learning version of RGN-res. *Autoregressive model* [10] is included to evaluate the performance of autoregressive models in few-shot learning. (11) Donà et al. present an autoregressive model designed to tackle forecasting according to diverse dynamics. *Gaussian Process* [11], [40] involves methods based on Gaussian processes. (12) GP-LVM [11] is a Gaussian Process Latent Variable Model. (13) GP [40] refers to the traditional Gaussian Process. *Meta-learning* [12], [13] involves methods based on meta-learning. (14) DyAd [12] is a meta-learning-based dynamic adaptation network. (15) Meta-N-Beats [13] is the meta-learning version of N-BEATS.

[1] Gravity-16 Dataset
[2] Mixed-physics Dataset
[3] MC Dataset

TABLE III
COMPARISON OF PREDICTION ACCURACY ON GRAVITY-16 AND MC

Dataset		Gravity-16		MC	
Model Category	Model	Dist↓	VPT-Dist↑	MAE↓	MSE↓
Ours	MetaGP	**2.06 (0.85)**	**0.99 (0.05)**	**1.21 (0.14)e-1**	**0.32 (0.03)e-1**
Meta-learning LVMs	meta-GRU-res	2.88 (1.45)	0.97 (0.07)	1.38 (0.18)e-1	0.46 (0.05)e-1
	meta-Node-res	6.10 (2.63)	0.80 (0.12)	2.24 (0.16)e-1	0.78 (0.12)e-1
	meta-RGN-res	6.97 (3.08)	0.76 (0.13)	2.41 (0.16)e-1	0.84 (0.13)e-1
System parameter as latent variable	GRU-res	10.4 (3.30)	0.61 (0.90)	3.35 (0.22)e-1	1.94 (0.22)e-1
	GRU-res finetune	9.35 (3.33)	0.66 (0.12)	2.64 (0.20)e-1	1.07 (0.18)e-1
	NODE	10.9 (3.32)	0.59 (0.08)	3.54 (0.25)e-1	2.72 (0.37)e-1
	NODE finetune	10.4 (3.23)	0.61 (0.09)	2.76 (0.25)e-1	1.15 (0.20)e-1
	RGN-res	11.2 (3.39)	0.58 (0.09)	3.63 (0.21)e-1	1.99 (0.29)e-1
	RGN-res finetune	10.0 (3.36)	0.62 (0.11)	2.74 (0.25)e-1	1.01 (0.19)e-1
	DVBF	45.3 (0.00)	0.00 (0.00)	4.57 (0.49)e-1	3.21 (0.51)e-1
	DVBF finetune	45.3 (0.00)	0.00 (0.00)	4.48 (0.46)e-1	3.35 (0.55)e-1
	KVAE	4.81 (3.61)	0.57 (0.29)	3.22 (0.15)e-1	2.31 (0.43)e-1
System state as latent variable	meta-DKF	7.35 (3.26)	0.70 (0.25)	3.93 (0.53)e-1	2.51 (0.47)e-1
	DKF	7.39 (3.21)	0.69 (0.25)	3.98 (0.53)e-1	2.53 (0.45)e-1
	DKF finetune	7.51 (3.26)	0.69 (0.25)	3.95 (0.53)e-1	2.53 (0.45)e-1
	VRNN	23.1 (21.6)	0.51 (0.19)	4.24 (0.58)e-1	3.46 (0.51)e-1
	VRNN finetune	8.31 (11.6)	0.75 (0.19)	3.37 (0.62)e-1	2.71 (0.45)e-1
Autoregressive	Donà et al.	13.7 (3.05)	0.06 (0.15)	4.06 (0.19)e-1	3.34 (0.50)e-1
Gaussian Process	GP-LVM	4.63 (2.61)	0.74 (0.15)	2.32 (0.15)e-1	1.07 (0.23)e-1
	GP	8.81 (3.79)	0.58 (0.17)	4.70 (0.32)e-1	2.24 (0.49)e-1
Meta-Learning	DyAd	3.05 (0.98)	0.85 (0.12)	2.45 (0.19)e-1	0.63 (0.13)e-1
	Meta-N-BEATS	4.35 (1.23)	0.78 (0.14)	2.52 (0.18)e-1	0.74 (0.13)e-1

3) Implementation Details: All experiments are conducted on a server running Linux 18.04 with an Intel Xeon W-2155 CPU @ 3.30 GHz and two RTX GPUs with 24 GB memory.

We use the Adam [60] optimizer with a learning rate η in the range $[1e^{-5}, 1e^{-2}]$. The number of training epochs is chosen from $\{50, 100, 200, 500\}$, taking into account the capacities of the datasets. The basic kernel set includes four simple kernel functions: SE, RQ, LIN, and PER. The candidate patch-size is chosen from all the common divisors of the length of the time window. The order R is chosen among $\{1, 2, 3, 4, 5\}$. The moving step Δ is choosen among $\{1, 2, 4, 8\}$. We use grid search [61] to select optimal hyperparameters from among the candidates described above. Furthermore, we tune the hyperparameters carefully based on the recommendations provided for the baselines.

B. Comparison of Prediction Accuracy

We proceed to study prediction accuracy on Gravity-16 and MC. Standard deviations are provided in parentheses. The best results are highlighted in boldface.

Table III shows a comparison of the average prediction accuracy and standard deviation across five repeated experiments for MetaGP and the baselines. The results show that MetaGP achieves state-of-the-art prediction accuracy on Gravity-16 and MC.

In few-shot learning, autoregressive methods exhibit lower prediction accuracy compared to meta-learning methods. This is attributed to the autoregressive nature of the methods, which necessitates a large number of samples to establish correspondences between historical and forecasting horizons within latent spaces. Consequently, autoregressive methods struggle to generalize model parameters to new tasks using only a limited number of samples.

Both methods, whether utilizing system parameters as latent states or system states as latent states, perform worse than the meta-learning methods on Gravity-16. However, on the MC, the approach employing system parameters as latent states outperforms the approach using system states as latent states. This is attributed to the presence of long-term dependencies in the MC time series when compared to the Gravity-16 image sequences. Using parameters as priors provides global information, which helps alleviate the challenges faced by sequential LVMs in handling long-term dependencies. In addition, the fine-tuned versions of the baselines show slight improvements in prediction accuracy over their non-fine-tuned counterparts. This suggests that although simple fine-tuning methods can mitigate the issue of insufficient samples, the effect of fine-tuning is minimal.

Furthermore, Gaussian process-based methods achieve limited prediction accuracy due to their reliance on a single prior. Next, as in the case of LVMs, meta-learning-based methods achieve limited prediction accuracy due to their limitations in capturing long-term dependencies.

The meta-learning LVMs exhibit prediction accuracies that are second only to that of MetaGP on the two datasets, indicating that the meta-learning method benefits sequential LVMs in few-shot learning. Notably, the decline in prediction accuracy of meta-learning LVMs is more pronounced on MC than on Gravity-16. This suggests that the Gaussian processes help MetaGP capture long-term dependencies.

C. Ablation Study

We conduct an ablation study on Gravity-16 and MC to assess the effectiveness of the components in MetaGP. The results are shown in Table IV, where MetaGP w/o Att refers to MetaGP with the attention mechanism replaced by the GRU-res structure, MetaGP w/o GP denotes MetaGP with the Gaussian process replaced by a traditional sequential LVM structure, MetaGP

TABLE IV
ABLATION STUDY OF METAGP

Dataset		Gravity-16		MC	
Model	Horizon F	Dist↓	VPT-Dist↑	MAE↓	MSE↓
MetaGP	12	**2.06 (0.85)**	**0.99 (0.05)**	**1.21 (0.14)e-1**	**0.32 (0.03)e-1**
	24	**2.34 (0.94)**	**0.98 (0.06)**	**1.26 (0.19)e-1**	**0.34 (0.04)e-1**
	48	**3.02 (1.59)**	**0.95 (0.08)**	**1.39 (0.21)e-1**	**0.37 (0.04)e-1**
MetaGP w/o Att	12	2.14 (1.24)	0.98 (0.07)	1.27 (0.15)e-1	0.39 (0.04)e-1
	24	2.85 (1.52)	0.95 (0.09)	1.36 (0.24)e-1	0.41 (0.05)e-1
	48	3.39 (1.82)	0.94 (0.09)	1.53 (0.25)e-1	0.45 (0.05)e-1
MetaGP w/o GP	12	2.54 (1.43)	0.96 (0.08)	1.33 (0.22)e-1	0.41 (0.05)e-1
	24	3.98 (2.32)	0.85 (0.10)	2.03 (0.26)e-1	1.06 (0.18)e-1
	48	5.72 (3.39)	0.79 (0.13)	2.43 (0.30)e-1	1.91 (0.22)e-1
MetaGP w/o CVW	12	3.31 (0.93)	0.92 (0.02)	1.81 (0.25)e-1	0.85 (0.15)e-1
	24	3.44 (1.21)	0.89 (0.03)	1.88 (0.25)e-1	1.08 (0.17)e-1
	48	3.70 (1.24)	0.88 (0.03)	2.05 (0.27)e-1	1.10 (0.17)e-1
MetaGP w/o KAS	12	3.05 (1.56)	0.97 (0.03)	2.04 (0.28)e-1	1.05 (0.16)e-1
	24	3.15 (1.68)	0.95 (0.05)	2.09 (0.28)e-1	1.07 (0.11)e-1
	48	3.42 (1.83)	0.92 (0.06)	2.26 (0.30)e-1	1.10 (0.13)e-1
MetaGP w/o NAS	12	2.25 (1.03)	0.98 (0.06)	1.22 (0.15)e-1	0.37 (0.04)e-1
	24	2.41 (1.08)	0.97 (0.07)	1.29 (0.21)e-1	0.40 (0.05)e-1
	48	3.48 (1.65)	0.94 (0.07)	1.46 (0.23)e-1	0.43 (0.06)e-1

w/o CVW indicates MetaGP without cross-variable weights and using instead a unified kernel function for all variables, MetaGP w/o KAS represents MetaGP without the meta-learning structure and using instead an SE kernel, and MetaGP w/o NAS is MetaGP without the neural architecture search-based meta-learning strategy, employing instead a greedy strategy. We fix the historical horizon at 8 but vary the forecasting horizon among 12, 24, and 48 to assess the performance of MetaGP at capturing long-term dependencies.

MetaGP w/o Att shows a slight decrease in prediction accuracy compared to MetaGP on both datasets, indicating that the attention mechanism is better at capturing multi-scale information in time series than in recurrent structures. However, the slight improvement also suggests that addressing the key challenges of few-shot time series forecasting cannot be achieved solely by replacing the encoder of the model.

MetaGP w/o GP suffers a notable decrease in prediction accuracy as forecasting horizon F increases. This indicates that Gaussian processes are crucial for capturing long-term dependencies. Gaussian processes incur less accumulation of losses compared to traditional sequential LVMs. Moreover, compared to w/o Att, w/o GP suffers a much larger decline in prediction accuracy as the forecasting horizon increases, highlighting that the kernel functions of Gaussian processes play a critical role in capturing long-term dependencies. In contrast, the attention mechanism contributes primarily to strong feature representation rather than to modeling long-term temporal structures in few-shot learning.

MetaGP w/o CVW shows lower prediction accuracy than MetaGP, indicating that cross-variable weights play a crucial role in capturing correlations among multiple variables, which is beneficial for few-shot learning. Additionally, we observe that MetaGP w/o CVW has a smaller prediction standard deviation than other baselines, suggesting that the model, using a fixed kernel function, experiences underfitting. Its predictions are smoother but also less accurate.

MetaGP w/o KAS shows a more pronounced decrease in accuracy on MC compared to Gravity-16. Unlike the gravity system's image sequences, MC features more complex temporal dynamics that consist of different combinations of temporal patterns. Removing KAS limits MetaGP to using a single kernel function to capture such intricate temporal dynamics, thereby reducing MetaGP's ability to effectively model such complex information.

MetaGP w/o NAS shows prediction accuracy similar to MetaGP because both methods tend to converge to similar kernel functions during kernel search. However, the greedy strategy carries the risk of getting stuck in local optima. For instance, if a task lacks a trend pattern but the basic kernel function set includes a trend kernel, the model might retain this kernel in the initial training rounds due to reduced loss. Thus, the greedy strategy may preserve unnecessary components.

In summary, all components contribute to improving the prediction accuracy of MetaGP, demonstrating the efficacy of the framework and its components.

D. Few-Shot Learning and Scalability

We assess the scalability of MetaGP by examining the impact of varying the number of few-shot learning training samples on its prediction accuracy, as well as its prediction accuracy on systems with different dynamic complexities.

1) Study of Few-Shot Learning: We select and compare the prediction accuracy of MetaGP and the second-best meta-learning method, meta-GRU-res, on Gravity-16 and MC in few-shot learning. We set the number of few-shot learning samples to 1, 5, 10, and 15, and observe the changes in prediction accuracy.

As shown in Table V, MetaGP outperforms the baseline in prediction accuracy across all shots. Additionally, as expected, the prediction accuracy of MetaGP and the baseline increase with the number of training samples. When the number of training samples reaches 5, the rate of increase in prediction accuracy for both methods slows down. This indicates that MetaGP's performance is not significantly affected even with a small number of training samples. MetaGP's scalability and flexibility at predicting time series with any given few-shot size.

TABLE V
FEW-SHOT LEARNING OF METAGP

Dataset		Gravity-16		MC	
Model	Shot	Dist↓	VPT-Dist↑	MAE↓	MSE↓
MetaGP	1	**6.38 (2.60)**	**0.74 (0.07)**	**2.93 (0.21)e-1**	**1.39 (0.22)e-1**
	5	**2.64 (1.04)**	**0.97 (0.06)**	**1.55 (0.19)e-1**	**0.30 (0.05)e-1**
	10	**2.24 (0.89)**	**0.98 (0.05)**	**1.28 (0.15)e-1**	**0.36 (0.04)e-1**
	15	**2.06 (0.85)**	**0.99 (0.05)**	**1.21 (0.14)e-1**	**0.32 (0.03)e-1**
meta-GRU-res	1	10.6 (3.40)	0.60 (0.10)	3.31 (0.51)e-1	2.49 (0.37)e-1
	5	3.49 (1.89)	0.94 (0.10)	1.75 (0.24)e-1	1.72 (0.20)e-1
	10	3.08 (1.58)	0.96 (0.08)	1.52 (0.20)e-1	1.64 (0.16)e-1
	15	2.88 (1.45)	0.97 (0.07)	1.38 (0.18)e-1	0.46 (0.05)e-1

TABLE VI
COMPARISON OF PREDICTION ACCURACY ON THE MIXED-PHYSICS DATASET

Dataset	Bouncing Ball		Pendulum		Mass-spring	
Model	Dist↓	VPT-Dist↑	Dist↓	VPT-Dist↑	Dist↓	VPT-Dist↑
MetaGP	**2.45 (1.22)**	**0.97 (0.05)**	**1.51 (1.64)**	**0.97 (0.08)**	**0.15 (0.07)**	**1.00 (0.00)**
meta-GRU-res	4.39 (2.50)	0.89 (0.13)	1.74 (1.99)	0.96 (0.10)	0.18 (0.09)	1.00 (0.00)
GRU-res	4.61 (2.68)	0.89 (0.14)	3.57 (3.86)	0.87 (0.20)	0.25 (0.15)	1.00 (0.00)
GRU-res finetune	4.37 (2.49)	0.90 (0.13)	3.55 (4.05)	0.87 (0.20)	0.16 (0.08)	1.00 (0.00)
meta-DKF	7.28 (3.55)	0.72 (0.26)	3.49 (2.96)	0.93 (0.18)	0.54 (0.29)	1.00 (0.00)
DKF	9.78 (3.45)	0.38 (0.26)	6.55 (4.21)	0.66 (0.35)	0.86 (0.48)	1.00 (0.00)
DKF finetune	9.87 (3.37)	0.37 (0.26)	6.03 (4.12)	0.68 (0.35)	0.83 (0.47)	1.00 (0.00)
KAVE	10.0 (3.01)	0.42 (0.34)	4.12 (4.26)	0.70 (0.34)	0.52 (0.51)	1.00 (0.00)
Donà et al.	13.8 (2.97)	0.06 (0.17)	15.8 (2.31)	0.03 (0.08)	5.60 (2.79)	0.61 (0.45)
GP-LVM	4.49 (2.73)	0.90 (0.15)	1.89 (1.87)	0.93 (0.12)	0.24 (0.10)	1.00 (0.00)
DyAd	4.86 (2.54)	0.82 (0.20)	2.03 (2.14)	0.85 (0.15)	0.26 (0.13)	1.00 (0.00)

2) Study on Dynamics of Different Complexity: We proceed to study prediction accuracy on Mixed-physics to assess the ability of MetaGP and the baselines at capturing varying meta-knowledge patterns in systems with different dynamic complexities. Jiang et al. [3] find that Bouncing Ball in the Mixed-physics dataset exhibits more complex dynamics than do Pendulum and Mass-spring. This is because Bouncing Ball's underlying gravity system is more complex than those underlying Pendulum and Mass-spring.

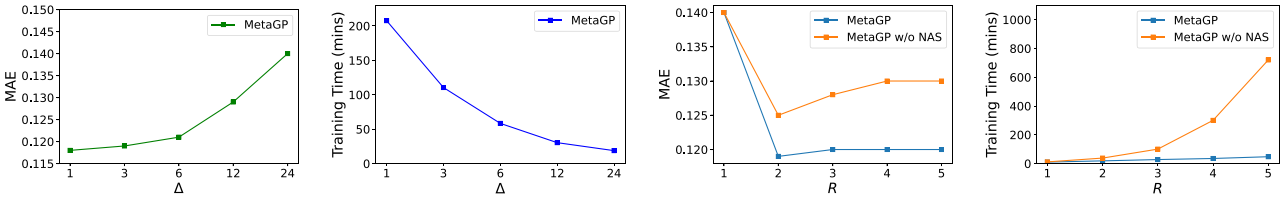
We align our experimental setup with that of Jiang et al. [3] and select one representative baseline from each of the 6 categories: meta-GRU-res, KAVE, DKF, Donà et al, GL-LVM, and DyAd. In addition, we include derived fine-tuned and meta-learning versions of these baselines, if applicable: GRU-res, GRU-res finetune, DKF finetune, and meta-DKF. The findings, in Table VI, show that MetaGP achieves state-of-the-art prediction accuracy on both the complex Bouncing Ball and the simpler Pendulum and Mass-spring.

All methods exhibit higher prediction accuracy on data from simpler systems than from complex systems, indicating that as system complexity increases, the difficulty of capturing varying patterns also increases. Additionally, the fine-tuned and meta-learning versions of the baselines outperform the original versions. To mitigate the impact of the few-shot learning, we compare MetaGP with fine-tuned and meta-learning versions of the baselines. We observe that the performance improvement of MetaGP on complex systems is much better than that on simpler systems, suggesting that the meta-learning component can transform varying patterns into meta-knowledge and capture complex dynamics effectively. This indicates that MetaGP can scale to data from systems with complex dynamics.

We proceed to consider the impact of changes in MetaGP's hyperparameters Δ and R on its prediction accuracy and training time. All experiments are conducted on MC. Additionally, we use the full meta-training set and set the historical horizon L to 48 while keeping the forecasting horizon F at 12 to obtain more experimental results.

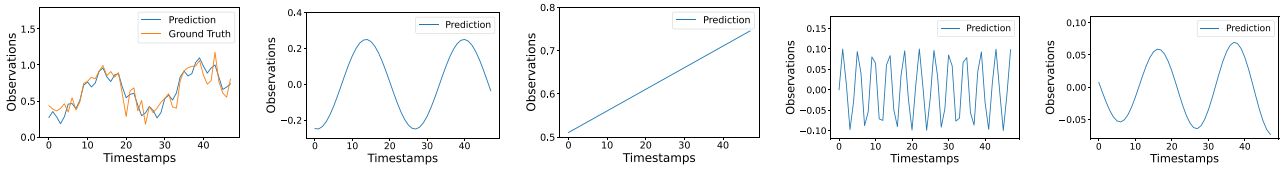
As shown in Fig. 7(a), the MAE of MetaGP increases with Δ. This is because the overlap and information density between input subsequences decreases, which decreases the accuracy. Moreover, as shown in Fig. 7(b), the training time of MetaGP decreases with a larger Δ, as using fewer input sequences accelerates training. We can thus use hyperparameter Δ to balance between the computational resources used and the prediction accuracy.

Next, on the M4 dataset, we vary depth R to assess the sensitivity to R. MetaGP w/o NAS employs a greedy strategy for meta-learning, and we use it as a baseline. Due to the high time complexity of MetaGP w/o NAS, we set Δ to 24, as this reduces the training time. Fig. 7(c) shows that MetaGP and MetaGP w/o NAS achieve their highest prediction accuracy when $R = 2$ or $R = 3$. Although a larger R yields more high-order hybrid patterns for fitting the training data, this also increases the risk of overfitting. Finally, we report the training time of KAS and greedy kernel search. Fig. 7(d) shows that MetaGP has a lower training time than MetaGP w/o NAS. Therefore, MetaGP's KAS, which employs neural architecture search, not only avoids local optima and maintains a prediction accuracy that is at least as good as that of the greedy strategy, it also reduces the time complexity of the greedy strategy. We recommend setting R to 2 or 3 to maintain a short training time without jeopardizing prediction accuracy.



(a) MAE Vary with Δ (b) Training Time Vary with Δ (c) MAE Vary with R (d) Training Time Vary with R

Fig. 7. Hyperparameter sensitivity.

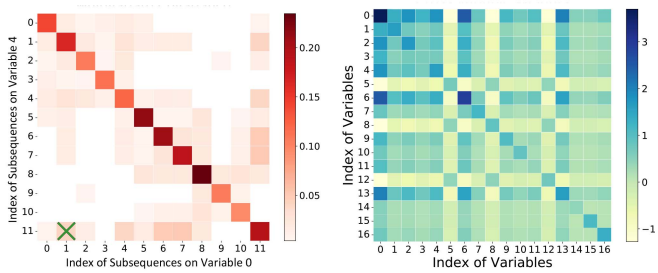


(a) Prediction vs. ground truth (b) Long-term seasonality captured by the PER kernel (c) Trend captured by the LIN kernel (d) Mixed short-term dependency and seasonality (e) Mixed trend and long-term seasonality

Fig. 8. Visualization of meta-knowledge patterns.

TABLE VII
ROBUSTNESS STUDY ON GRAVITY-16 AND MC

Dataset	Ratio	1	3	5
Gravity-16	MetaGP	**2.14 (0.88)**	**2.32 (0.91)**	**2.54 (0.93)**
	meta-GRU-res	3.05 (1.49)	3.34 (1.55)	3.59 (1.63)
MC	MetaGP	**1.27 (0.15)e-1**	**1.30 (0.15)e-1**	**1.37 (0.17)e-1**
	meta-GRU-res	1.43 (0.19)e-1	1.49 (0.20)e-1	1.61 (0.23)e-1



(a) Multivariate Covariance Matrix (b) Cross-variable Weights

Fig. 9. Visualization of two types of correlations.

E. Robustness Study

To evaluate the robustness of MetaGP, we introduce outliers with a magnitude of five times the standard deviation into the Gravity-16 and MC datasets. The outlier ratios are set to 1%, 3%, and 5%, representing the proportion of samples containing outliers relative to the total number of samples. We compare with meta-GRU-res, the most accurate method among the baselines.

As shown in Table VII, the accuracy of both methods drops as the outlier ratio increases. However, MetaGP consistently achieves the best accuracy. This indicates that although few-shot learning is affected by outliers, MetaGP maintains superior robustness.

F. Visualization

Next, we visualize different meta-knowledge patterns and multivariate correlations captured by MetaGP.

1) Visualization of Meta-Knowledge Patterns: We extract a sample from MC that contains different patterns to assess the ability of KAS to model and capture different meta-knowledge patterns.

Fig. 8(a) shows MetaGP's predictions and the corresponding ground truth for the sample. MetaGP can fit the forecasting horizon well. This prediction is generated by the meta-knowledge learned by KAS. We rank the generated kernel functions by their weight and present the predictions produced by the top four functions; see Fig. 8(b)–(e).

Fig. 8(b)–(c) show two simple patterns: long-term seasonality and trend, modeled and captured by the PER and LIN kernels learned by KAS. Next, Fig. 8(d)–(e) show two complex patterns: mixed short-term dependency and seasonality, and mixed trend and long-term seasonality, modeled and captured by the SE $\times$ PER and LIN $\times$ RQ kernels learned by KAS.

2) Visualization of Correlations: There are two types of correlations between variables: one is the specific correlation between variables of two input sequences, as exemplified in Fig. 9(a); the other is the generalized correlation that applies broadly to one type of data, as exemplified in Fig. 9(b).

MetaGP utilizes a multivariate covariance matrix to capture the correlations between each two input sequences. An example matrix that captures the correlation between variables 0 and 4 is shown in Fig. 9(a), where darker colors indicate higher positive correlations and lighter colors indicate higher negative correlations. We observe that the correlations between different input sequences can be captured. For example, input sequence

1 of variable 0 has a positive correlation with input sequence 11 of variable 4; see the cross in Fig. 9(a). The values in the covariance matrix reflect the true correlations between specific samples and can be applied in scientific analyse.

Then, we show cross-variable weights in Fig. 9(b). The higher an absolute weight value is, the higher the importance of the corresponding variable. The clear differences between the cross-variable weights indicate that the dynamics between variables in a time series are highly complex, making it difficult to describe them with a single dynamic function. This underscores the necessity of this module. Moreover, cross-variable weights reflect a global and stable correlation between variables. This reflects the stable numerical relationship between physical quantities.

VI. Conclusion and Future Work

This study addresses the need for effective few-shot time series forecasting, especially given the limited sample sizes in fields like physics and biology. We present the meta-learning Gaussian process latent variable model, MetaGP, which effectively addresses the challenges of capturing long-term dependencies and explicitly modeling of diverse meta-knowledge. By employing a Gaussian process, MetaGP maintains strong correlations across distant observations, overcoming the limitations of existing sequential latent variable models. Additionally, MetaGP's Kernel Association Search (KAS) component enhances its ability to capture meta-knowledge and correlations within multivariate time series, improving both prediction accuracy and interpretability. Extensive experiments show that MetaGP is capable of state-of-the-art performance on simulated and real-world biological data, confirming its effectiveness at few-shot forecasting. The latent state architecture of MetaGP requires re-execution of the meta-learning process to discover meta-knowledge when transferring to different domains. However, as knowledge across related domains may share common structures, this re-execution may be unnecessary. In future research, we aim to improve the meta-learning by leveraging the generalization and knowledge summarization capabilities of large language models. This integration is expected to improve the cross-domain deployability of MetaGP while increasing both efficiency and accuracy.

References

[1] L. Sun et al., "SciEval: A multi-level large language model evaluation benchmark for scientific research," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 19053–19061.

[2] Z. Zhong, K. Zhou, and D. Mottin, "Benchmarking large language models for molecule prediction tasks," 2024, arXiv: *2403.05075*.

[3] X. Jiang, R. Missel, Z. Li, and L. Wang, "Sequential latent variable models for few-shot high-dimensional time-series forecasting," in *Proc. Int. Conf. Learn. Representations*, 2023.

[4] A. Botev, A. Jaegle, P. Wirnsberger, D. Hennes, and I. Higgins, "Which priors matter? Benchmarking models for learning latent dynamics," in *Proc. Neural Inf. Process. Syst. Track Datasets Benchmarks*, 2021.

[5] M. Fraccaro, S. Kamronn, U. Paquet, and O. Winther, "A disentangled recognition and nonlinear dynamics model for unsupervised learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 3601–3610.

[6] M. Karl, M. Soelch, J. Bayer, and P. Smagt, "Deep variational bayes filters: Unsupervised learning of state space models from raw data," in *Proc. Int. Conf. Learn. Representations*, 2017.

[7] Y. Nie, N. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *Proc. Int. Conf. Learn. Representations*, 2023.

[8] J. Chung, K. Kastner, L. Dinh, K. Goel, A. Courville, and Y. Bengio, "A recurrent latent variable model for sequential data," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 2980–2988.

[9] R. Krishnan, U. Shalit, and D. Sontag, "Structured inference networks for nonlinear state space models," in *Proc. AAAI Conf. Artif. Intell.*, 2017, pp. 2101–2109.

[10] J. Doná, J. Franceschi, S. Lamprier, and P. Gallinari, "PDE-driven spatiotemporal disentanglement," in *Proc. Int. Conf. Learn. Representations*, 2021.

[11] V. Lalchand, A. Ravuri, and N. Lawrence, "Generalised Gaussian process latent variable models (GPLVM) with stochastic variational inference," 2022, arXiv: *2202.12979*.

[12] R. Wang, R. Walters, and R. Yu, "Meta-learning dynamics forecasting using task inference," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 21640–21653.

[13] B. Oreshkin, D. Carpov, N. Chapados, and Y. Bengio, "Meta-learning framework with applications to zero-shot time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 9242–9250.

[14] S. Li et al., "Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 5243–5253.

[15] S. Liu et al., "Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting," in *Proc. Int. Conf. Learn. Representations*, 2021.

[16] P. Chen et al., "Pathformer: Multi-scale transformers with Adaptive Pathways for Time Series Forecasting," in *Proc. Int. Conf. Learn. Representations*, 2024.

[17] Y. Zhang and J. Yan, "Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting," in *Proc. Int. Conf. Learn. Representations*, 2023.

[18] X. Wang, T. Zhou, Q. Wen, J. Gao, B. Ding, and R. Jin, "CARD: Channel aligned robust blend transformer for time series forecasting," in *Proc. Int. Conf. Learn. Representations*, 2024.

[19] Y. Liu et al., "ITransformer: Inverted transformers are effective for time series forecasting," in *Proc. Int. Conf. Learn. Representations*, 2024.

[20] P. Montero-Manso, G. Athanasopoulos, R. Hyndman, and T. Talagala, "FFORMA: Feature-based forecast model averaging," *Int. J. Forecasting*, vol. 36, pp. 86–92, 2020.

[21] X. Chen, C. Ge, M. Wang, and J. Wang, "Supervised contrastive few-shot learning for high-frequency time series," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 7069–7077.

[22] Y. Wu et al., "Dynamic Gaussian mixture based deep generative model for robust forecasting on sparse multivariate time series," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 651–659.

[23] S. Zhong, V. Souza, G. Baker, and A. Mueen, "Online few-shot time series classification for aftershock detection," in *Proc. SIGKDD Conf. Knowl. Discov. Data Mining*, 2023, pp. 5707–5716.

[24] L. Bai, L. Yao, S. Kanhere, Z. Yang, J. Chu, and X. Wang, "Passenger demand forecasting with multi-task convolutional recurrent neural networks," in *Proc. Pacific-Asia Conf. Knowl. Discov. Data Mining*, 2019, pp. 29–42.

[25] J. Wang, Z. Wang, J. Li, and J. Wu, "Multilevel wavelet decomposition network for interpretable time series analysis," in *Proc. SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 2437–2446.

[26] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, "Graph wavenet for deep spatial-temporal graph modeling," 2019, arXiv: *1906.00121*.

[27] D. Campos et al., "Unsupervised time series outlier detection with diversity-driven convolutional ensembles–extended version," 2021, arXiv: *2111.11108*.

[28] B. Lim, S. Arık, N. Loeff, and T. Pfister, "Temporal fusion transformers for interpretable multi-horizon time series forecasting," *Int. J. Forecasting*, vol. 37, pp. 1748–1764, 2021.

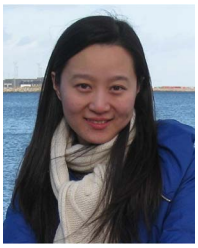
[29] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 22419–22430.

[30] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 11106–11115.

[31] L. Bai, L. Yao, C. Li, X. Wang, and C. Wang, "Adaptive graph convolutional recurrent network for traffic forecasting," in *Proc. Adv. in Neural Inf. Process. Syst.*, 2020, pp. 17804–17815.

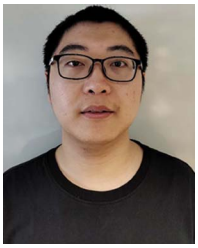
[32] R. Cirstea, T. Kieu, C. Guo, B. Yang, and S. Pan, "EnhanceNet: Plugin neural networks for enhancing correlated time series forecasting," in *Proc. Int. Conf. Data Eng.*, 2021, pp. 1739–1750.
[33] D. Hallac, Y. Park, S. Boyd, and J. Leskovec, "Network inference via the time-varying graphical Lasso," in *Proc. SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2017, pp. 205–213.
[34] D. Cao et al., "Spectral temporal graph neural network for multivariate time-series forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 17766–17778.
[35] A. Vaswani et al., "Attention is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.
[36] R. Cirstea, C. Guo, B. Yang, T. Kieu, X. Dong, and S. Pan, "Triformer: Triangular, variable-specific attentions for long sequence multivariate time series forecasting," in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, pp. 1994–2001.
[37] A. Wilson and R. Adams, "Gaussian process kernels for pattern discovery and extrapolation," in *Proc. Int. Conf. Mach. Learn.*, 2013, pp. 1067–1075.
[38] A. Tong and J. Choi, "Discovering latent covariance structures for multiple time series," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6285–6294.
[39] D. Oktay, N. McGreivy, J. Aduol, A. Beatson, and R. Adams, "Randomized automatic differentiation," in *Proc. Int. Conf. Learn. Representations*, 2021.
[40] C. Williams and C. Rasmussen, *Gaussian Processes for Machine Learning*. Cambridge, MA, USA: MIT Press, 2006.
[41] A. Wilson, *Covariance Kernels for Fast Automatic Pattern Discovery and Extrapolation With Gaussian Processes*. Cambridge, U.K.: Univ. Cambridge, 2014.
[42] C. Rasmussen, "Gaussian processes in machine learning," *Summer Sch. Mach. Learn.*, vol. 3176, pp. 63–71, 2003.
[43] D. Duvenaud, "Automatic model construction with Gaussian processes," in *Comput. Biological Learn. Lab.*, Cambridge, U.K.: University of Cambridge, 2014.
[44] M. Al-Shedivat, A. Wilson, Y. Saatchi, Z. Hu, and E. Xing, "Learning scalable deep kernels with recurrent structure," *J. Mach. Learn. Res.*, vol. 18, pp. 2850–2886, 2017.
[45] D. Duvenaud, J. Lloyd, R. Grosse, J. Tenenbaum, and G. Zoubin, "Structure discovery in nonparametric regression through compositional kernel search," in *Proc. Int. Conf. Mach. Learn.*, 2013, pp. 1166–1174.
[46] Y. Motai, "Kernel association for classification and prediction: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 26, no. 2, pp. 208–223, Feb. 2015.
[47] T. Hastie, R. Tibshirani, J. Friedman, and J. Friedman, *The elements of statistical learning: Data mining, inference, and prediction*. Berlin, Germany: Springer, 2009.
[48] R. Grosse, R. Salakhutdinov, W. Freeman, and J. Tenenbaum, "Exploiting compositionality to explore a large space of model structures," in *PRoc. Conf. Uncertainty Artif. Intell.*, 2012, pp. 306–315.
[49] J. Lloyd, D. Duvenaud, R. Grosse, J. Tenenbaum, and Z. Ghahramani, "Automatic construction and natural-language description of nonparametric regression models," in *Proc. AAAI Conf. Artif. Intell.*, 2014, pp. 1242–1250.
[50] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," in *Proc. Int. Conf. Learn. Representations*, 2019.
[51] X. Wu, D. Zhang, C. Guo, C. He, B. Yang, and C. Jensen, "AutoCTS: Automated correlated time series forecasting," *VLDB Endowment*, vol. 15, pp. 971–983, 2021.
[52] G. Parra and F. Tobar, "Spectral mixture kernels for multi-output Gaussian processes," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6684–6693.
[53] A. Wilson, Z. Hu, R. Salakhutdinov, and E. Xing, "Stochastic variational deep kernel learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 2594–2602.
[54] A. Wilson, Z. Hu, R. Salakhutdinov, and E. Xing, "Deep kernel learning," in *Proc. Conf. Artif. Intell. Statist.*, 2016, pp. 370–378.
[55] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M4 competition: Results, findings, conclusion and way forward," *Int. J. Forecasting*, vol. 34, pp. 802–808, 2018.
[56] H. Dau et al., "The UCR time series archive," *IEEE/CAA J. Automatica Sinica*, vol. 6, pp. 1293–1305, 2019.
[57] F. Zamora-Martinez, P. Romeu, P. Botella-Rocamora, and J. Pardo, "Online learning of indoor temperature forecasting models towards energy efficiency," *Energy Buildings*, vol. 83, pp. 162–172, 2014.
[58] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," 2017, arXiv: *1707.01926*.
[59] T. Guo, T. Lin, and N. Antulov-Fantulin, "Exploring interpretable LSTM neural networks over multi-variable data," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2494–2504.
[60] D. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Representations*, 2015.
[61] F. Pontes, G. Amorim, P. Balestrassi, A. Paiva, and J. Ferreira, "Design of experiments and focused grid search for neural network parameter optimization," *Neurocomputing*, vol. 186, pp. 22–34, 2016.
[62] A. Ismail, M. Gunady, H. Corrada Bravo, and S. Feizi, "Benchmarking deep learning interpretability in time series predictions," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 6441–6452.
[63] C. Holt, "Forecasting seasonals and trends by exponentially weighted moving averages," *Int. J. Forecasting*, vol. 20, pp. 5–10, 2004.
[64] M. Jankowiak, G. Pleiss, and J. Gardner, "Deep sigma point processes," in *Proc. Conf. Uncertainty Artif. Intell.*, 2020, pp. 789–798.
[65] H. Salimbeni and M. Deisenroth, "Doubly stochastic variational inference for deep Gaussian processes," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 4591–4602.
[66] A. Damianou and N. Lawrence, "Deep Gaussian processes," in *Proc. Conf. Artif. Intell. Statist.*, 2013, pp. 207–215.
[67] E. Bonilla, K. Chai, and C. Williams, "Multi-task Gaussian process prediction," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2007, pp. 153–160.
[68] A. Wilson, D. Knowles, and Z. Ghahramani, "Gaussian process regression networks," in *Proc. Int. Conf. Mach. Learn.*, 2012, pp. 1139–1146.
[69] M. Titsias and M. Lázaro-Gredilla, "Spike and slab variational inference for multi-task and multiple kernel learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2011, pp. 2339–2347.
[70] C. Guarnizo, M. Álvarez, and A. Orozco, "Indian buffet process for model selection in latent force models," in *Proc. Conf. Iberoamerican Congr. Pattern Recognit.*, 2015, pp. 635–642.
[71] E. Schulz, J. Tenenbaum, D. Duvenaud, M. Speekenbrink, and S. Gershman, "Compositional inductive biases in function learning," *Cogn. Psychol.*, vol. 99, pp. 44–79, 2017.
[72] H. Kim and Y. Teh, "Scaling up the automatic statistician: Scalable structure discovery using Gaussian processes," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2018, pp. 575–584.
[73] X. Lu, J. Gonzalez, Z. Dai, and N. Lawrence, "Structured variationally auto-encoded optimization," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 3267–3275.
[74] S. Sun, G. Zhang, C. Wang, W. Zeng, J. Li, and R. Grosse, "Differentiable compositional kernel learning for Gaussian processes," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4828–4837.
[75] T. Kieu et al., "Robust and explainable autoencoders for unsupervised time series outlier detection," in *Proc. Int. Conf. Data Eng.*, 2022, pp. 3038–3050.
[76] Y. Zhang, P. Tiňo, A. Leonardis, and K. Tang, "A survey on neural network interpretability," *IEEE Trans. Emerg. Top. Comput. Intell.*, vol. 5, no. 5, pp. 726–742, Oct. 2021.
[77] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 27268–27286.
[78] A. Wilson and H. Nickisch, "Kernel interpolation for scalable structured Gaussian processes (KISS-GP)," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1775–1784.

Yunyao Cheng received the master's degree in computer science from the University of Texas at Arlington, USA, in 2020. He is currently working toward the PhD degree with the Department of Computer Science, Aalborg University, Denmark. His research interests include machine learning and spatio-temporal data mining.



Chenjuan Guo received the PhD degree in computer science from the University of Manchester, U.K., in 2011. She is a professor with East China Normal University, China. She was previously with Aalborg University, Denmark. Her research interests include spatial-temporal data management, heterogeneous data management, and data analytics.

Kaixuan Chen received the PhD degree in computer science from the University of New South Wales, Australia, in 2020. She is an assistant professor with Aalborg University, Denmark. Her research interests include data mining, deep learning, and sensor-based human activity recognition.



Kai Zhao received the bachelor's degree from Peking University, China and the master's degree in computer science from the Beijing University of Posts and Telecommunications and his bachelor degree from Peking University, China, in 2018 and 2021, respectively. He is currently working toward the PhD degree with the Department of Computer Science, Aalborg University, Denmark. His research interests include machine learning, graph learning, and spatio-temporal data mining.

Bin Yang (Senior Member, IEEE) received the PhD degree in computer science from Fudan University, China. He is a professor with East China Normal University, China. He was previously with Aalborg University, Denmark, Aarhus University, Denmark, and MaxPlanck-Institut fur Informatik, Germany. His research interests include machine learning and data management.

Jiandong Xie received the PhD degree from the university of electronic science and technology of China (UESTC) in 2020 and has been working in Huawei as a data analyst since. His current research interests include time series analysis algorithms and AI-driven query optimizers.

Christian S. Jensen (Fellow, IEEE) received the PhD degree from Aalborg University, Denmark, where he is a professor. His research concerns data analytics and management with a focus on temporal and spatiotemporal data. He is a fellow of the ACM, and he is a member of the Academia Europaea, the Royal Danish Academy of Sciences and Letters, and the Danish Academy of Technical Sciences.

Feiteng Huang received the PhD degree from Sichuan University in 2014. He is a technical expert with the Huawei Cloud Database Innovation Lab. He joined Huawei, where he has primarily focused on research and development in the areas of storage and databases, including distributed storage systems, key-value (KV) storage, document databases, and time-series databases.

Kai Zheng (Senior Member, IEEE) received the PhD degree in computer science from the University of Queensland, in 2012. He is a professor of computer science with the University of Electronic Science and Technology of China. He has been working in the area of spatial-temporal databases, uncertain databases, social-media analysis, in-memory computing, and blockchain technologies.